

Procedures and the Stack

Procedure calls, return linkage, stack frames, parameter passing, and AAPCS64 calling conventions in AArch64

Topic 8 Outline

Topic Objective

Analysis of subroutine linkage, procedure abstraction, stack organization, parameter passing rules, and register preservation contracts under AArch64 AAPCS64.

- **Section 1:** Procedure Abstraction and the Call-Return Model
- **Section 2:** Subroutine Linkage (b1, ret, and Link Register x30)
- **Section 3:** Stack Organization, Activation Records, and Frame Pointers
- **Section 4:** Parameter Passing and AAPCS64 Calling Conventions
- **Section 5:** Summary and Worked Self-Test Exercises

The Need for Modular Procedures

Modular Software Design

Programs are broken down into discrete functions or procedures to promote code reuse, simplify debugging, and manage software complexity.

- Requires a standardized mechanism to transfer control to a subroutine and back to the exact caller.
- Must support passing inputs (arguments), receiving outputs (return values), and protecting local variables.
- Governed strictly by platform-specific Application Binary Interfaces (ABIs).

Procedure Abstraction and Call-Return Model

The Subroutine Call-Return Lifecycle

Executing a procedure involves several distinct operational phases:

- ➊ **Setup:** Caller places function arguments into designated parameter registers (x0–x7).
- ➋ **Transfer:** Branch to function label via `bl`, saving the return address in x30 (`lr`).
- ➌ **Prologue:** Allocate stack space and save registers that must be preserved.
- ➍ **Execution:** Perform the core computation of the subroutine.
- ➎ **Epilogue:** Restore saved registers, deallocate stack space, and return via `ret`.

Caller vs. Callee Responsibilities

Maintaining modular integrity requires dividing responsibilities between functions:

- **The Caller Responsibilities:**

- Prepares and passes arguments in registers `x0–x7` (or on the stack if exceeding 8).
- Invokes the subroutine using `bl`, which saves the return address in `x30` (`lr`).
- Preserves any caller-saved volatile registers (`x0–x15`) across the call if their values are still needed.

- **The Callee Responsibilities:**

- Allocates local stack frame space and preserves callee-saved registers (`x19–x29`) and `x30` if executing nested calls.
- Computes the result and places return values into `x0`.
- Restores saved registers, deallocates stack memory, and returns via `ret`.

Mapping C Functions to AArch64 Labels

C Function Example

```
int find_max(int a, int b) {  
    int max = (a > b) ? a : b;  
    return max;  
}
```

AArch64 Translation Principle

- AArch64 assembly does not have native function declarations—only symbolic labels.
- Arguments are passed in x0 and x1.
- The return value is placed in x0.

Subroutine Linkage (BL, RET, and Link Register)

Branch with Link: The bl Instruction

Syntax and Semantics

```
bl function_label
```

Performs a subroutine call by branching to the target label while simultaneously saving the return address (the instruction immediately following the bl) into the Link Register (x30 / lr).

- Essential for enabling nested function calls.
- The Link Register acts as a hardware-supported return address holder for the current scope.

Returning from Subroutines: The `ret` Instruction

Syntax and Semantics

`ret`

Unconditionally branches back to the address held in the Link Register (`x30`), resuming execution in the calling function.

- Equivalent to executing an indirect branch: `br x30`.
- Modern microarchitectures optimize `ret` using hardware return-stack predictors (unlike legacy `mov pc, lr`).

Handling Nested Subroutine Calls

What happens if Function A calls Function B using `bl`?

- The `bl` to Function B overwrites the Link Register (`x30`) with Function B's return address, erasing Function A's return address!
- **The Solution:** Before calling another function, a callee must save its incoming `x30` value onto the stack during its prologue.

Nested Call Assembly Pattern

```
outer_func:
    stp x29, x30, [sp, #-16]!    // Save outer return address (x30) on stack
    bl inner_func                // Overwrites x30 with inner return address
    ldp x29, x30, [sp], #16      // Restore original outer return address
    ret                          // Correctly returns to caller of outer_func
```

Stack Organization, Activation Records, and Frame Pointers

The Stack Memory Structure

The system stack provides temporary workspace, register storage, and local variables:

- **Last-In, First-Out (LIFO):** Operates like a stack of plates; data is pushed and popped strictly at the top.
- **Full-Descending Model:** The stack grows downward in memory toward lower addresses. The Stack Pointer (`sp` / `x31`) points directly to the last valid item pushed.
- **Strict Alignment:** The stack pointer must maintain 16-byte alignment at all times to prevent hardware faults.

Stack vs. Heap Growth

The stack grows downward while the heap grows upward, maximizing available runtime memory efficiency within process address spaces.

Concrete Stack Allocation Example

Allocating local storage and saving register context during a function prologue:

Prologue Instruction Breakdown

```
sub sp, sp, #32    // Step 1: Decrement stack pointer by 32 bytes
stp x29, x30, [sp, #16] // Step 2: Store Frame Pointer and Link Register
add x29, sp, #16    // Step 3: Establish new Frame Pointer reference
```

- **Pre-indexed Alternative:** Compilers often combine allocation and storage using a single writeback instruction: `stp x29, x30, [sp, #-32]!` followed by `mov x29, sp`.
- Maintains strict 16-byte stack boundary alignment required by AArch64 hardware specifications.

Frame Pointers and Stack Activation Records

Each active function maintains a localized stack frame (activation record):

- **Frame Pointer (x29):** Points to the base of the current stack frame, anchoring local variables and saved registers.
- **Prologue Stack Growth:** `stp x29, x30, [sp, #-16]!` reserves space and saves both frame pointer and link register.
- **Epilogue Restoration:** `ldp x29, x30, [sp], #16` restores them and releases stack space.

Accessing Local Variables via Frame Pointer

Once the prologue establishes the Frame Pointer (x29), local variables are accessed using fixed offsets:

- **Positive Offsets:** Relative to x29 to access saved registers (e.g., [x29, #16]).
- **Negative Offsets:** Relative to x29 to access local function variables stored higher in the stack frame (e.g., [x29, #-8]).

Example Function Epilogue Cleanup

```
ldp x29, x30, [sp], #32    // Restore FP and LR, post-increment stack pointer
ret                        // Return control back to caller via lr
```


An AArch64 Stack Frame Layout

- **Higher Addresses:** Caller's stack frame data.
- **Saved Frame Pointer (x29):** Links back to previous frame.
- **Saved Link Register (x30):** Return address to caller.
- **Local Variables:** Scratch space for function variables.
- **Lower Addresses:** Current Stack Pointer (sp).

Stack Growth Direction (↓)

Higher Memory Addresses

Caller Frame

Saved Frame Pointer (x29)

Saved Link Register (x30)

Local Variables / Buffers

Recursion and Stack Overflow Limits

Functions that call themselves (recursion) use the exact same call-return mechanics:

- Every recursive invocation allocates a brand new stack frame to hold its unique local variables, x29, and x30.
- **The Danger:** Because the stack memory space is finite (often capped at 8 MB on Linux), infinite or excessively deep recursion will consume all available space.
- This results in a **Stack Overflow** (segmentation fault), crashing the program without any compiler warning.

Parameter Passing and AAPCS64 Calling Conventions

Passing Function Arguments (x0–x7)

To maximize speed, AArch64 passes arguments through general-purpose registers:

- **Registers x0 through x7:** Designated for passing the first 8 integer/pointer arguments to a subroutine.
- Using registers is exponentially faster than pushing and popping variables from main memory.

Order Convention

Argument 1 goes in x0, Argument 2 in x1, up to Argument 8 in x7.

Handling More Than 8 Arguments

If a function requires 9 or more arguments, the register file is insufficient:

- **Overflow Arguments:** Any arguments beyond the first 8 must be pushed onto the stack by the *caller* before executing the `bl` instruction.
- The callee then reads these extra arguments directly from the caller's stack frame using positive offsets from the Stack Pointer.
- **Cleanup:** After the callee returns, the caller is responsible for adjusting the stack pointer back up to remove the pushed arguments.

Returning Function Values

Subroutines return computation results back to callers via designated registers:

- **Registers x0 and x1:** Used for returning integer or pointer values.
- x0 holds the primary return value (equivalent to a function's `return` statement in C).
- Larger structures or multi-word values utilize indirect result pointers passed via x8.

Example: Returning an Integer Calculation

```
calc_sum:  
    add x0, x1, x2    // Add arguments in x1 and x2, place result in x0  
    ret               // Return control; caller reads result from x0
```

Register Preservation Conventions (AAPCS64)

Functions share a common register file, requiring strict rules on preservation:

- **Caller-Saved (Volatile)** — x0–x15: Can be freely overwritten by a callee. If a caller needs values in these registers to survive a function call, it must save them beforehand.
- **Callee-Saved (Non-Volatile)** — x19–x29: Must be preserved across calls. If a callee modifies any of these registers, it must save their original values to the stack in its prologue and restore them in its epilogue.

A Complete Function Prologue and Epilogue

Example of a callee function that allocates local stack space and preserves registers:

```
my_func:
    stp x29, x30, [sp, #-32]!    // Allocate 32 bytes, save FP & LR
    mov x29, sp                 // Set up new frame pointer
    str x19, [x29, #16]         // Save callee-saved register x19
    // ... Function Body Execution ...
    ldr x19, [x29, #16]         // Restore x19
    ldp x29, x30, [sp], #32     // Restore FP & LR, deallocate stack
    ret                         // Return to caller
```


Summary & Self-Test

Topic 8 Comprehensive Summary

- **Call-Return Model:** Procedures rely on standardized linkage to pass control, arguments, and return values safely.
- **Link Register:** `bl` stores return addresses in `x30` (`lr`), and `ret` uses them to jump back.
- **Stack Organization:** The stack grows downward, managed via `sp` and anchored by frame pointer `x29`.
- **Parameter Passing:** Registers `x0`–`x7` handle arguments; `x0` returns results.
- **AAPCS64 Conventions:** Strict caller-saved vs. callee-saved register preservation rules maintain modular safety.

Self-Test: Questions 1 & 2

Question 1: The BL Instruction

What two actions are performed simultaneously when the processor executes a `bl target_label` instruction?

Question 2: Parameter Limits

How many integer or pointer arguments can be passed to a subroutine using registers alone under the AAPCS64 standard, and where do extra arguments go?

Self-Test: Solutions 1 & 2

Solution 1

It saves the return address (the next sequential instruction address) into the Link Register (x30) and branches execution to the target label.

Solution 2

Up to 8 arguments can be passed via registers x0 through x7. Any additional arguments exceeding 8 must be pushed onto the stack by the caller.

Self-Test: Questions 3 & 4

Question 3: Callee-Saved Registers

Why must a subroutine preserve callee-saved registers (x19–x29) if it intends to modify them during execution?

Question 4: Nested Function Calls

Why is it mandatory for a function that calls another subroutine to save the Link Register (x30) onto the stack during its prologue?

Self-Test: Solutions 3 & 4

Solution 3

To protect data belonging to the caller. If a callee overwrites a callee-saved register without restoring it, the calling function's data would become corrupted upon return.

Solution 4

Executing a nested `bl` instruction overwrites the Link Register (`x30`) with a new return address, destroying the original return address needed to get back to the original caller.

Self-Test: Questions 5 & 6

Question 5: Stack Alignment

What byte alignment constraint does AArch64 enforce on the Stack Pointer (sp) during procedure calls, and why is it required?

Question 6: Frame Pointers

What role does the Frame Pointer (x29) play when managing activation records on the stack?

Self-Test: Solutions 5 & 6

Solution 5

The stack pointer must maintain strict 16-byte alignment to prevent hardware alignment faults and support wide load/store pair operations (ldp/stp).

Solution 6

The Frame Pointer anchors the base of the current stack frame, allowing local variables and saved registers to be referenced via fixed positive or negative offsets.

Review: AAPCS64 Register Conventions

Register Range	Role / Category	Preservation Rule
x0–x7	Argument passing & Return values	Caller-saved (Volatile)
x8–x15	Intra-procedure scratch registers	Caller-saved (Volatile)
x19–x29	Local variable storage	Callee-saved (Non-volatile)
x30 (lr)	Link Register (Return address)	Managed via Subroutine Call
x31 (sp)	Stack Pointer	Maintained 16-byte aligned

Topic 8 Source Notes

Source Reference Verification

- Subroutine linkage instructions (`bl`, `ret`) and stack pointer rules follow standard ARMv8/ARMv9 AArch64 architecture guidelines.
- Register preservation rules and parameter passing conventions conform strictly to the Procedure Call Standard for the ARM 64-bit Architecture (AAPCS64).

Wrap-Up and Next Steps

Transition to Topic 9

Building upon procedures, stack organization, and calling conventions, the next module examines:

- **Machine Instruction Encoding:** Deconstructing how abstract assembly instructions translate into binary machine code.
- **AArch64 Instruction Formats:** Analyzing fixed 32-bit instruction formats and bitfield layouts.
- **Data-Processing Instruction Encoding:** Inspecting register specifiers, size flags (sf), and immediate encodings.
- **Memory Instruction Encoding:** Studying base-register fields and PC-relative address generation.
- **Branch Instruction Encoding and Decoding:** Examining offset calculations and decoding machine-code instructions.